Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2115 - Programación como Herramienta para la Ingeniería

Extracción y visualización de datos

Profesor: Pablo Seisdedos

¿Qué es Web Scraping?

- Definición: Proceso de extracción de datos de páginas web de manera automatizada.
- Objetivo: Recopilar información que no está disponible directamente a través de APIs o bases de datos.
- Aplicaciones: Análisis de mercado, recopilación de datos para proyectos de machine learning, monitoreo de precios, entre otros.

Pasos Simples para Web Scraping

- Seleccionar el sitio web: Identificar la página que tiene los datos necesarios.
- Buscar los datos: Inspeccionar el HTML para localizar los elementos que contienen los datos deseados.
- Extraer los datos: Usar un lenguaje de programación como Python para extraer los datos.
- Guardar los datos: Almacenar la información en un formato útil, como CSV o JSON.
- Procesar y consultar los datos: Utilizar herramientas como Pandas para limpiar, transformar y analizar la información extraída.

Web Scraping en Python

- BeautifulSoup (BS4): Para analizar HTML y extraer información específica.
- Pandas: Para organizar, limpiar, analizar y también extraer datos de tablas HTML.

```python
# Extraer datos con pandas
import pandas as pd

url = 'https://ejemplo.com'
tables = pd.read_html(url)  # Extrae todas las tablas de la página
df = tables[0]  # Selecciona la primera tabla extraída
```

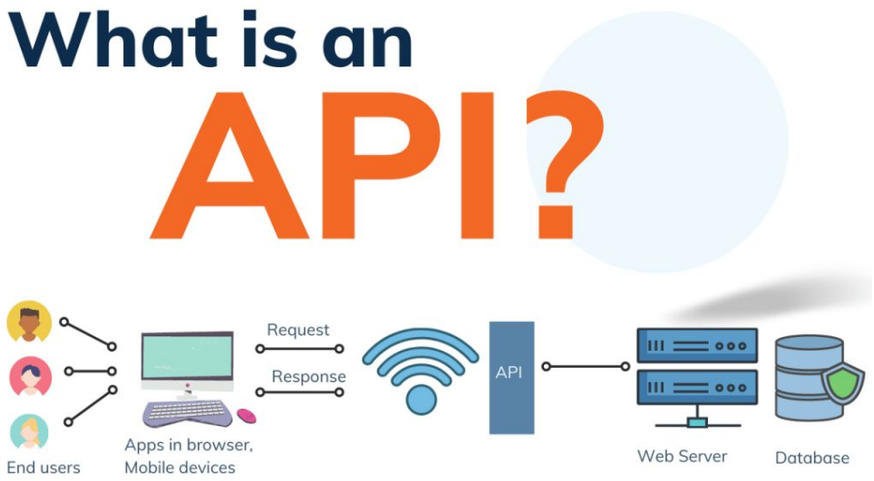
```python
import requests
from bs4 import BeautifulSoup
import pandas as pd

url = 'https://ejemplo.com'
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

# Extraer datos
titles = [title.get_text() for title in soup.find_all('h2')]
# Organizar y guadrar con pandas
df = pd.DataFrame(titles, columns=['Title'])
df.to_csv('output.csv', index=False)
```

Consideraciones Importantes

- Rate Limiting: No sobrecargar los servidores; usar pausas entre requests.
- Manejo de Errores: Usar manejo de excepciones para evitar que el script falle.
- Tiempo de Ejecución por Request: Usar time.sleep() para retrasar solicitudes y evitar bloqueos.
- Rotación de IPs y User Agents: Técnica para cambiar la IP y el agente de usuario en cada solicitud y evitar bloqueos por parte del sitio.

¿Qué es una API?

- Definición: Conjunto de reglas que permite que dos sistemas se comuniquen entre sí de forma estructurada.
- Objetivo: Facilitar el acceso a datos o funcionalidades de un sistema sin necesidad de interactuar directamente con su interfaz interna.
- Aplicaciones: Integración de servicios, automatización de procesos, acceso a datos en tiempo real (como clima, mapas, redes sociales), entre otros.



Tipos de APIs

- REST: Basada en HTTP. Usa métodos como GET, POST, PUT, DELETE.
- SOAP: Más estricto, basado en XML. Usado en servicios más antiguos.
- GraphQL: Permite hacer consultas más flexibles desde un solo endpoint.
- Webhooks: Permiten recibir datos en tiempo real ante ciertos eventos.

Ventajas de usar APIs

- Facilitan la integración entre aplicaciones.
- Permiten acceso a datos actualizados en tiempo real.
- Promueven la reutilización de servicios.
- Mejoran la escalabilidad de sistemas.

Componentes básicos de una request a una API

- URL: Dirección del endpoint al que se hace la solicitud (ej: https://api.openweathermap.org/data/2.5/weather).
- API Key: Clave que identifica al usuario y otorga acceso autorizado al servicio.
- Headers: Información adicional que acompaña la solicitud (ej: tipo de contenido, autorización).
- Parámetros: Datos enviados en la URL o el cuerpo para especificar la consulta (ej: ciudad, unidades, idioma).
- Respuesta (JSON): La mayoría de las APIs modernas devuelven datos en formato JSON, fácil de leer y procesar.

¿Cómo se construye la URL de una request a una API?

- Dominio base: la dirección del servidor de la API
    - Ej: https://api.openweathermap.org
- Endpoint: ruta específica para el recurso deseado
    - Ej: /data/2.5/weather
- Parámetros de consulta: valores clave para personalizar la respuesta
    - Ej: ?q=Santiago&units=metric&appid=TU_API_KEY

Diferencias entre Web Scraping y API

Característica	Web Scraping	API
Fuente de datos	HTML de páginas web	Respuesta estructurada (JSON/XML)
Robustez	Frágil ante cambios en el sitio	Más estable y documentada
Velocidad	Más lenta	Generalmente más rápida
Eficiencia de uso	Requiere procesar el HTML	Acceso directo a la información

¿Qué es una página web?

- Es un documento accesible por internet que puede mostrar texto, imágenes, gráficos, formularios y más.
- Se accede a través de un navegador (como Chrome o Firefox) y está escrita comúnmente en HTML, CSS y JavaScript.
- Tipos:
    - Estática: siempre muestra lo mismo.
    - Dinámica: cambia su contenido según el usuario o datos (ej: clima en tu ciudad).

¿Qué es dashboard?

- Es una página web interactiva que permite visualizar datos de forma clara y útil.
- Muestra gráficos, métricas, filtros y controles para explorar información.
- Se usa en áreas como:
    - Finanzas
    - Transporte
    - Clima
    - Salud
    - Educación

¿Qué es Dash?

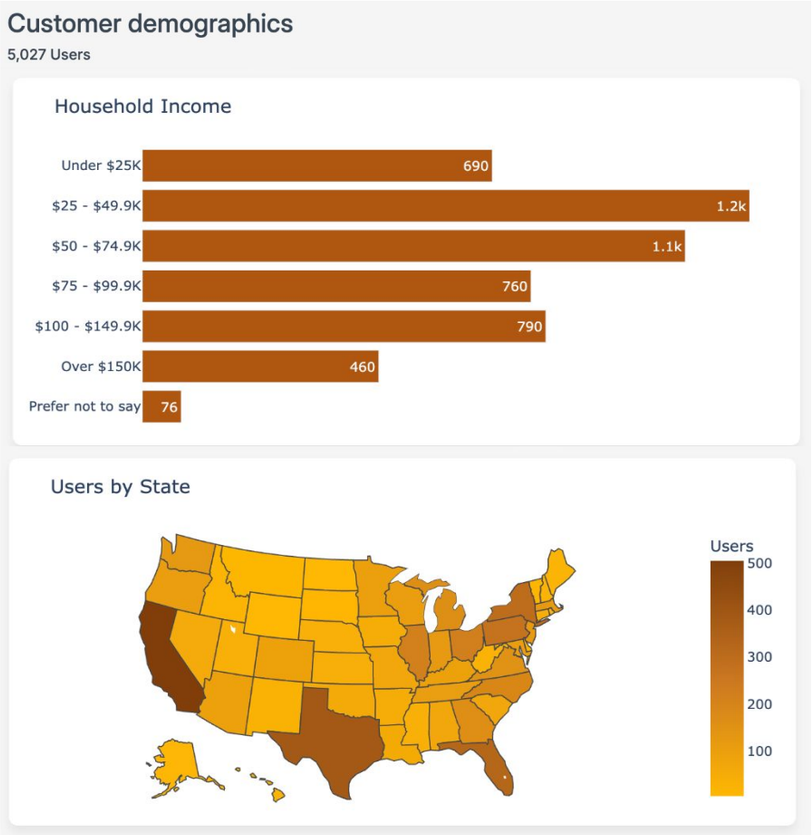
- Dash es un framework de Python para construir aplicaciones web interactivas sin necesidad de escribir código en HTML, CSS o JavaScript.
- Ideal para crear dashboards de visualización de datos y herramientas analíticas accesibles desde el navegador.
- Desarrollado por Plotly, combina lo mejor de Flask (web backend), React (interactividad) y Plotly (gráficos).

plotly | Dash

https://plotly.com/dash/

¿Qué se puede hacer con Dash?

- Visualizar datos en tiempo real desde APIs o bases de datos.
- Crear gráficos interactivos (líneas, barras, mapas, etc.).
- Filtrar información con sliders, dropdowns, checkboxes y más.
- Navegar entre diferentes páginas o secciones (tabs, URLs).
- Generar interfaces personalizadas sin salir de Python.



Componentes básicos de una app en Dash

- Layout:
    - Estructura visual de la app (títulos, textos, gráficos, menús, etc.). Se define con componentes de dash.html y dash.dcc.

  Callbacks:
    - Funciones que conectan la interacción del usuario con actualizaciones en la interfaz. Permiten hacer la app reactiva.
- Interactividad:
    - Dash permite usar elementos como dropdowns, sliders, checkboxes, tabs, y más, para que el usuario explore los datos a su ritmo.

¿Cómo funciona un callback en Dash?

- Un callback es una función en Python que se activa cuando el usuario interactúa con la app (por ejemplo, al seleccionar una ciudad o mover un slider).
- Cada callback tiene:
    - Input(s): Lo que el usuario cambia (ej: valor de un dropdown).
    - Output(s): Lo que se actualiza en la app (ej: contenido de un gráfico).
    - (Opcional) State: Valores actuales que no activan el callback, pero se pueden usar en el cálculo.
- Flujo de un callback:
    - Usuario interactúa con un componente →Dash detecta el cambio →Ejecuta la función del callback →Actualiza la parte correspondiente del layout

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2115 - Programación como Herramienta para la Ingeniería

Extracción y visualización de datos

Profesor: Pablo Seisdedos